var, Text Blocks and Small Language Wins

6 min read Updated 28 Jul 2026 Topic 48 of 65

Bookmark Mark done

Why it matters

These are the features you will use every day, and the ones where a bad answer reveals a stale mental model. "Does `var` make Java dynamically typed?" is asked precisely because the wrong answer is common — and the right answer, that inference happens entirely at compile time, connects to the type erasure and static typing themes from earlier chapters.

The concept, simply

`var` (Java 10) lets the compiler infer a local variable's type from its initialiser. The variable still has one fixed static type — nothing about the type system changed, only how much you have to type.

JAVA Copy

```java
var names = new ArrayList<String>();     // ArrayList<String>, inferred
var count = 10;                          // int
// names = new LinkedList<String>();     ✗ still statically typed
// var x;                                ✗ no initialiser, nothing to infe
// var y = null;                         ✗ null has no useful type
```

Text blocks (Java 15) are multi-line string literals delimited by triple quotes, with incidental indentation stripped automatically.

How it works

else that forms an API, because those types are contracts that should be written down.

The style guidance that matters: use `var` when the initialiser makes the type obvious, and write the type when it does not.

JAVA Copy

```
var users = new HashMap<String, List<Order>>();    // ✓ obvious, and saves
var order = orderService.find(id);                 // ✗ what does find() re
Optional<Order> order = orderService.find(id);     // ✓ worth spelling out
```

Text blocks compute the common indentation across all lines — including the closing delimiter — and strip it. Moving the closing `"""` changes the result, which is the one surprising rule.

Code examples

JAVA Copy

```
// ✗ Before: escaping, concatenation and manual newlines.
String json = "{
" +
              "  \"name\": \"Asha\",
" +
              "  \"role\": \"admin\"
" +
              "}";

// ✓ Text block — no escaping, and the shape is visible in the source.
String json = """
        {
          "name": "Asha",
          "role": "admin"
        }
        """;

// The closing delimiter sets the indentation baseline. Put it further lef
```

```
// and every line keeps that much leading whitespace.
// \ escapes the line break (no newline emitted); \s keeps a trailing spac
String sql = """
        SELECT id, name \
        FROM users \
        WHERE active = true""";        // one line, no trailing newline
```

The other small wins that come up:

JAVA Copy

```
// String methods (Java 11)
"  hi  ".strip();              // Unicode-aware — trim() only handles ASCI
"".isBlank();                  // true — whitespace-only counts
"ab".repeat(3);                // "ababab"
text.lines().filter( ... );    // Stream<String>, no split("
")

// Files (Java 11) — one line instead of a reader
String content = Files.readString(path);
Files.writeString(path, content);

// Collection factories (Java 9) — immutable, reject nulls
var list = List.of("a", "b");
var map = Map.of("k", 1, "j", 2);
var copy = List.copyOf(mutable);

// Formatted (Java 15) — reads better than String.format when chaining
"Hello %s, you are %d".formatted(name, age);

// Records + var + text blocks together is where modern Java starts to fee
// like a different language from Java 8.
```

Under the hood

`var` **is erased entirely at compile time.** The bytecode is identical to writing the type by hand — there is no runtime cost, no reflection difference, and no change to the type

One genuinely interesting consequence: `var` can infer types you *cannot write*. Assigning an anonymous class to a `var` gives the variable that anonymous type, so you can call methods declared in it — impossible with a named type. It is a curiosity rather than a technique, but it demonstrates that inference is doing real work.

Text blocks are ordinary strings after compilation — the compiler applies indentation stripping and escape processing and emits a normal string constant, which is interned like any other literal. There is no runtime component at all.

Common mistakes

- **Thinking `var` is dynamic typing.** It is compile-time inference; the type is fixed.
- **Using `var` when the type is not obvious.** It hurts readability at exactly the point where readers need help.
- **Trying `var` on fields or parameters.** Locals only, by design.
- `var x = new ArrayList<>();` — infers `ArrayList<Object>`, which is almost never intended.
- **Misplacing the closing `"""`.** It sets the indentation baseline.
- **Using `trim()` instead of `strip()`.** `trim()` predates Unicode and misses several whitespace characters.

Interview lens

> **Q: "Does `var` make Java dynamically typed?"**
> A: No. The compiler infers one specific static type from the initialiser and enforces it exactly as if you had written it — you cannot later assign an incompatible value. It is erased at compile time, so the bytecode is identical. It only reduces what you type, not what the type system does.

A: Local variables, loop variables and try-with-resources — never fields, parameters or return types, because those are API contracts that deserve explicit types. I use it when the initialiser makes the type obvious, like `new HashMap<String, List<Order>>()`, and write the type when the right-hand side is a method call whose return type is not evident.

Q: "How does indentation work in a text block?"
A: The compiler finds the minimum indentation across all lines, including the closing delimiter, and strips it. So moving the closing `"""` to the left preserves leading whitespace on every line. There is also `\` to suppress a line break and `\s` to keep a trailing space — both useful for wrapping long SQL without emitting newlines.

Q: "`strip()` or `trim()`?"
A: `strip()`. `trim()` dates from Java 1.0 and removes only characters up to U+0020, so it misses several Unicode whitespace characters. `strip()` is Unicode-aware, and there are `stripLeading` / `stripTrailing` variants plus `isBlank()`, which is the readable way to test for whitespace-only.

Cheat-card

- `var` = **compile-time inference**, locals only, erased in bytecode. **Not dynamic typing.**
- Use it when the initialiser makes the type obvious; write the type otherwise.
- `var x = new ArrayList<>()` → `ArrayList<Object>`. Watch the diamond.
- Text blocks: `"""`, incidental indentation stripped, **closing delimiter sets the baseline**.
- `\` suppresses the line break; `\s` keeps a trailing space.
- Java 11 strings: `strip` (Unicode) · `isBlank` · `repeat` · `lines()`.

QUICK CHECK — no marks, just for you.

1. Does var make Java dynamically typed?

- ◯ Yes, but only when the initialiser is a method call
- ◯ Yes — the variable's type can change at run time
- ◯ No — the compiler infers one specific static type and enforces it exactly as before; var is erased and the bytecode is identical
- ◯ Only for local variables inside lambdas

Check answer

2. In a text block, what determines how much leading whitespace is stripped?

- ◯ The indentation of the first line only
- ◯ Exactly four spaces are always removed
- ◯ Nothing is stripped; whitespace is preserved verbatim
- ◯ The minimum indentation across all lines INCLUDING the closing delimiter — so moving the closing """ left preserves more whitespace

Check answer

3. Why prefer strip() over trim()?

- ◯ trim() dates from Java 1.0 and only removes characters up to U+0020, so it misses several Unicode whitespace characters; strip() is Unicode-aware

- trim() was removed in Java 11
- strip() also removes punctuation

Check answer

Previous
Diagnosing Memory and Performance Problems

Next
Records and Sealed Types

LEARN MORE

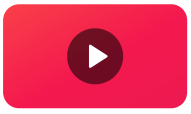
ARTICLES & DOCS

- Local-variable type inference (JEP 286)
  openjdk.org
- Text blocks (JEP 378)
  openjdk.org
- Local variable type inference style guidelines
  openjdk.org

VIDEOS



- "var and text blocks" — video explainers
  YouTube



- var and text blocks interview questions
  YouTube

Hand-picked references for this lesson.